



DAYANANDA SAGAR COLLEGE OF ENGINEERING
(An Autonomous Institute Affiliated to VTU, Belagavi, Accredited by NAAC with 'A' Grade)
Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru-560111

Department of Computer Science & Engineering (Cyber Security)

CLOUD SECURITY LAB (22CY63)

By-ShrestaMangala L (DSCE)

PLATFORM: Amazon Web Service

AWS Billing User Guide

AWS Billing and Cost Management provides a suite of features to help you set up your billing, retrieve and pay invoices, and analyse, organize, plan, and optimize your costs.

See the following overview of features to help you manage your cloud finances.

Features of AWS Billing and Cost Management

Topics

- Billing and payments
- Cost analysis
- Cost organization
- Budgeting and planning
- Savings and commitments

Billing and payments

Understand your monthly charges, view and pay invoices, and manage preferences for billing, invoices, tax, and payments.

Cost analysis

Analyse your costs, export detailed cost and usage data, and forecast your spending.

Cost organization

Organize your costs across teams, applications, or end customers.

Budgeting and planning

Estimate the cost of a planned workload, and create budgets to track and control costs.

Savings and commitments

Optimize resource usage and use flexible pricing models to lower your bill.

Creation of budget (User Guide) with alerts.

Step1: Click on Billing and cost management, choose Budget

Step2: Choose budget type. Click on Customize a budget to set parameters specific to your use case. You can customize the time period, the start month, and specific accounts.

Step3: Budget name, Provide a descriptive name for this budget.

Step4: Set budget amount with specific period, budgeting renewal method amount budget type.

Step5: Enter your budgeted amount

Step6: Budget scope. Click on All AWS services (Recommended).

Step7: Add on alert threshold with mail id (For one budget you can create up to 10 alerts). **Step8:** Create budget-finish

Experiment 1

How to create VPC, Subnet, Internet gateway and Routing Table in AWS Services.

Steps for creations of VPC, Subnet, Internet gateway and Routing Table

Virtual Private Cloud (VPC) is a logically isolated section of the AWS Cloud where you can launch AWS resources in a virtual network, allowing you to control your own IP address ranges, subnets, and security settings.

Subnet is a range of IP addresses within a Virtual Private Cloud (VPC) that you can use to launch AWS resources like EC2 instances, and they are restricted to a single Availability Zone

Route table contains a set of rules, called routes, that determine where network traffic from your subnet or gateway is directed, essentially acting as a map for routing within your Virtual Private Cloud (VPC).

Internet Gateway (IGW) is a horizontally scaled, redundant, and highly available VPC component that facilitates two-way communication between your VPC and the internet, enabling resources in public subnets with public IP addresses to connect to and receive connections from the internet.

Amazon RDS for SQL Server is a managed service that simplifies deploying, operating, and scaling SQL Server databases in the cloud, handling tasks like provisioning, patching, and backups, while you focus on application development.

1. Create a VPC

1. Go to **VPC Dashboard**
2. Click **Create VPC**
3. Choose:

VPC only

4. Fill details:

Name: MyVPC

IPv4 CIDR block: 10.0.0.0/16

5. Click **Create VPC**

This CIDR gives you ~65,000 IP addresses

2. Create a Subnet

1. In VPC Dashboard → **Subnets**
2. Click **Create subnet**
3. Select your VPC (MyVPC)
4. Enter:

Name: PublicSubnet

Availability Zone: choose one (e.g., ap-south-1a)

IPv4 CIDR: 10.0.1.0/24

5. Click **Create subnet**

/24 gives 256 IPs

3. Create an Internet Gateway (IGW)

1. Go to **Internet Gateways**
2. Click **Create internet gateway**
3. Name it: MyIGW
4. Click **Create**

Attach IGW to VPC

1. Select the IGW
2. Click **Actions** → **Attach to VPC**
3. Choose MyVPC
4. Attach

4. Create a Route Table

1. Go to **Route Tables**
2. Click **Create route table**
3. Enter:
Name: PublicRouteTable
VPC: MyVPC
4. Create

5. Add Route to Internet

1. Select PublicRouteTable
2. Go to **Routes tab**
3. Click **Edit routes** → **Add route**
4. Enter:
Destination: 0.0.0.0/0 (all traffic)
Target: **Internet Gateway** → **MyIGW**
5. Save

This allows internet access

6. Associate Subnet with Route Table

1. In Route Table → **Subnet Associations**
2. Click **Edit subnet associations**
3. Select PublicSubnet
4. Save

7. Enable Auto-Assign Public IP

1. Go to **Subnets**
2. Select PublicSubnet
3. Click **Actions** → **Edit subnet settings**
4. Enable:
Auto-assign public IPv4 address
5. Save

Take a screenshot of creation of VPC, Subnet, Internet gateway and Routing Table

S3 Bucket

An S3 bucket is a storage container provided by Amazon S3 (Simple Storage Service), part of Amazon Web Services (AWS). You can think of it as a folder in the cloud where you store and organize your data. Here are the key concepts:

Basics of an S3 Bucket:

Name: Globally unique across all AWS accounts.

Region: Created in a specific AWS region.

Objects: Files stored in a bucket. Each object has a key (its unique identifier), data, and metadata.

Folders: Not real directories, just part of the object key path to organize data.

Common Use Cases:

Static website hosting

Backup and restore

Data archiving

Media storage (images, videos, etc.)

Big data analytics storage

Key Features:

Versioning: Keep multiple versions of an object.

Access control: Managed via IAM policies, bucket policies, and ACLs.

Lifecycle policies: Automate transition to cheaper storage classes or deletion.

Encryption: Support for server-side and client-side encryption

Experiment 2

How to create a object using S3 bucket and how the move the object to folder.

Step 1: Create a bucket. Bucket name should globally unique.

Step 2: Choose one file from the desktop.

Step 3: Disable the bucket public option.

Step 4: Enabled the bucket versioning and save it.

Step5: Upload one file from the desktop.

Step 6: Create a folder. Click on the file, go to action and move it.

Step 7: Click on browse option, Select the folder and choose the destination.

Take a screenshot of both File and Folder you created.

Experiment 3

How to Host a Static Website using S3 Bucket

Step1: Create a bucket.

Step2: Upload the folder.

Step3: disable the bucket public option.

Step4: enable static website option.

Step5: enable ACL permission.

Step6: make it public ACL for all the folder object.

Step7: Click on html folder and host it.

Step 8: Check whether it is working properly or not.

1. Create an S3 Bucket

1. Go to the **AWS Management Console**
2. Navigate to **S3**
3. Click **Create bucket**
4. Enter:

Bucket name → must be globally unique (e.g., my-static-site-123)

Region → choose nearest to your users

5. **Uncheck:**

“Block all public access”  (important for public website)

6. Acknowledge the warning and create the bucket

2. Upload Your Website Files

Upload your static files:

index.html (required)

error.html (optional)

CSS, JS, images

Steps:

1. Open your bucket
2. Click **Upload**
3. Add files → Upload

3. Enable Static Website Hosting

1. Go to **Properties** tab
2. Scroll to **Static website hosting**
3. Click **Edit**
4. Enable:

Static website hosting

5. Set:

Index document → index.html

Error document → error.html (optional)

6. Save changes

You'll get a **website endpoint URL** (like: <http://bucket-name.s3-website-region.amazonaws.com>)

4. Make Files Public (Bucket Policy)

Without this, your site won't be accessible.

1. Go to **Permissions** tab
2. Scroll to **Bucket Policy**
3. Add this policy:

5. Disable Block Public Access (if not already)

1. Go to **Permissions**
2. Click **Edit** under Block public access
3. Uncheck all → Save

6. Test Your Website

Open the **S3 website endpoint URL** in your browser.

Take a screenshot of uploading folder page, and static website which displayed on chrome (Any 3)

Experiment 4

How to Create a Virtual Machine in AWS using PuTTY Software.

To create a virtual machine in **Amazon Web Services**, you use the EC2 service, and then connect to it using **PuTTY**.

Step 1: Launch an EC2 Instance (VM)

1. Open the **AWS Management Console**
2. Go to **EC2 Dashboard**
3. Click **Launch Instance**
4. Configure:

Name: MyVM

AMI (OS): Choose Amazon Linux or Ubuntu

Instance type: t2.micro (free tier eligible)

5. Create or select a **Key Pair** (download .pem file)
6. Under Network settings:
Allow **SSH (port 22)** from your IP
7. Click **Launch Instance**

Step 2: Convert .pem Key to .ppk (for PuTTY)

PuTTY requires a .ppk key instead of .pem.

1. Open **PuTTYgen** (comes with PuTTY)
2. Click **Load** → select your .pem file
3. Click **Save private key**
4. Save as .ppk

Step 3: Connect to EC2 Using PuTTY

1. Open **PuTTY**
2. In **Host Name**, enter:

ec2-user@<Public-IP>

(Example: [ec2-user@3.110.xxx.xxx](#))

3. Go to:

Connection → SSH → Auth

Browse and select your .ppk file

4. Click **Open**

Step 4: Login to Your VM

For Amazon Linux:

Username: ec2-user

For Ubuntu:

Username: ubuntu

You are now successfully connected to your virtual machine

Take a screenshot of EC2 instance, Putty login window and Ubuntu login server page.

Experiment 5

How to create AWS Services using Template.

Creating AWS services using templates typically involves using AWS CloudFormation, which allows you to define your infrastructure as code. This approach has several benefits, including consistency, repeatability, and easier management of resources. Below is a guide on how to create AWS services using CloudFormation templates, along with the benefits of using this method.

Step1: Select a Region. For Example: Mumbai

Step2: Go to Cloud Formation in AWS

Step3: Save code using yaml/JSON formatted file

AWS::CloudFormation::Template

Description: Create a VPC, two subnets, and an EC2 instance.

Resources:

Create VPC

MyVPC:

Type: AWS::EC2::VPC

Properties:

CidrBlock: 10.0.0.0/16

EnableDnsSupport: true

EnableDnsHostnames: true

Tags:

Key: Name

Value: MyVPC

Create Internet Gateway

MyInternetGateway:

Type: AWS::EC2::InternetGateway

AttachGateway:

Type: AWS::EC2::VPCGatewayAttachment

Properties:

VpcId: !Ref MyVPC

InternetGatewayId: !Ref MyInternetGateway

Create Public Route Table

PublicRouteTable:

Type: AWS::EC2::RouteTable

Properties:

VpcId: !Ref MyVPC

Create Route to Internet

PublicRoute:

Type: AWS::EC2::Route

Properties:

RouteTableId: !Ref PublicRouteTable

DestinationCidrBlock: 0.0.0.0/0

GatewayId: !Ref MyInternetGateway

Subnet 1

Subnet1:

Type: AWS::EC2::Subnet

Properties:

VpcId: !Ref MyVPC

CidrBlock: 10.0.1.0/24

AvailabilityZone: !Select [0, !GetAZs "]

Subnet 2

Subnet2:

Type: AWS::EC2::Subnet

Properties:

VpcId: !Ref MyVPC

CidrBlock: 10.0.2.0/24

AvailabilityZone: !Select [1, !GetAZs "]

Associate Subnet 1 with Public Route Table

Subnet1RouteTableAssociation:

Type: AWS::EC2::SubnetRouteTableAssociation

Properties:

SubnetId: !Ref Subnet1

RouteTableId: !Ref PublicRouteTable

Create Security Group

MySecurityGroup:

Type: AWS::EC2::SecurityGroup

Properties:

GroupDescription: Allow SSH and HTTP

VpcId: !Ref MyVPC

SecurityGroupIngress:

- IpProtocol: tcp

FromPort: 22

ToPort: 22

CidrIp: 0.0.0.0/0

- IpProtocol: tcp

FromPort: 80

ToPort: 80

CidrIp: 0.0.0.0/0

Launch EC2 Instance in Subnet 1

MyEC2Instance:

Type: AWS::EC2::Instance

Properties:

InstanceType: t2.micro

ImageId: ami-0e35ddab05955cf57 # Replace with a valid AMI ID for your region

SubnetId: !Ref Subnet1

SecurityGroupIds:

- !Ref MySecurityGroup

KeyName: my-key-pair2 # Replace with your key pair name

Step4: Create Stack. Step5: Upload a template file, Upload your template directly to the console.(code should be in JSON or YAML formatted file).

Step6: Give stack name and submit (create it)

Outputs:

VPCId:

Description: VPC ID

Value: !Ref MyVPC

Subnet1Id:

Description: Subnet 1 ID

Value: !Ref Subnet1

Subnet2Id:

Description: Subnet 2 ID

Value: !Ref Subnet2

InstanceId:

Description: EC2 Instance ID

Value: !Ref MyEC2Instance

Successfully created all services.

Take a screenshot of EC2 instances/VPC/Subnet/Internet Gateway (any 3)

Experiment 6

How to Auto Stop Virtual Instances in AWS Cloud to Save the cost during non-working hours.

To automatically stop AWS EC2 instances during non-working hours, you can use tools like AWS Instance Scheduler or AWS Lambda functions. These solutions allow you to set schedules for starting and stopping instances, significantly reducing costs by ensuring they are not running when not needed.

Select a Region. For example North Virginia US -East

Step 1 Create a EC2 instance

Step 2 Create a IAM Role with Lambda services along with EC2Full access Permission

Step 3 Copy the Instance ID (Virtual Machine ID) from EC2 Instance

Step 4 Create a Lambda function along with the Code and replace the instance ID and region in the code.

```
import boto3

import botocore

def lambda_handler(event, context):

    ec2 = boto3.client('ec2', region_name='us-east-1') # Corrected region name
    instance_id = "i-0cb771a7775f3909f" # Replace with your EC2 Instance ID

    try:

        response = ec2.stop_instances(InstanceIds=[instance_id])

        return {

            'statusCode': 200,

            'body': f"Stopping instance {instance_id}. Response: {response}"

        }

    except botocore.exceptions.ClientError as e:

        return {

            'statusCode': 400,

            'body': f"Error stopping instance {instance_id}: {str(e)}"

        }

    except Exception as e:

        return {

            'statusCode': 500,
```



```
'body': f'An unexpected error occurred: {str(e)}'
```

```
}
```

Step 5 Click on the Deploy button to save the code

Step 6 Go to the configuration option in Lambda function and change the execution time from 3 seconds to 1 min

Step 7 Click on the Test option in Lambda function and create a Test event

Step 8 Click on the Test

Step 9 Go to the EC2 console and check your EC2 instance is stopped or not

Take a screenshot of both EC2 instance START(Running) and EC2 instances STOPPED

Experiment 7

How to create Relational Database Service in AWS.

1. Log in to AWS Console

Go to the **AWS Management Console**

Sign in with your account

2. Navigate to RDS

Search for **RDS** in the services bar

Click on **Amazon RDS**

3. Click “Create Database”

You’ll see two options:

Easy Create (quick setup)

Standard Create (full control – recommended for learning)

4. Choose Database Engine

Pick one depending on your use case:

MySQL

PostgreSQL

MariaDB

Oracle Database

Microsoft SQL Server

For beginners: MySQL or PostgreSQL is easiest.

5. Choose a Template

Free Tier (if eligible)

Dev/Test

Production

6. Configure Settings

Fill in:

DB Instance Identifier (name)

Master username & password

7. Instance Configuration

Choose instance type (e.g., db.t3.micro for free tier)

8. Storage Settings

Select storage type (General Purpose SSD recommended)

Allocate storage (e.g., 20 GB)

9. Connectivity

Choose:

VPC (default is fine for beginners)

Public access: **Yes** (if you want to connect externally)

Configure security group (allow your IP for database port like 3306 for MySQL)

10. Additional Configuration

Optional but useful:

Initial database name

Backup settings

Monitoring options

11. Create Database

Click **Create Database**

Wait ~5–10 minutes for provisioning.

Take a screenshot of database(created)